1

Essential Math Toolkit

Before we dive into AI and reinforcement learning, let us make sure that we are comfortable with the mathematical language we will be using. Think of this chapter as your toolkit: we are going to explain every symbol, every operation, and every concept that you will see in this book.

Don't worry if you're not a math expert! We'll start from the basics and build up slowly. By the end of this chapter, you'll be comfortable with probability and what it really means, the mysterious "given" symbol |, why logarithms are so useful, what "expected value" means, and how to read mathematical notation.

Key promise: Every single symbol you see later in the book will make sense because we'll explain it here first!

Probability: The Language of Uncertainty

What Is Probability?

Probability is just a fancy way of measuring "how likely is something to happen?" We use numbers between 0 and 1:

- 0 = Impossible (will never happen)
- 0.5 = Maybe (50/50 chance)
- 1 = Certain (will definitely happen)

You can also think of it as percentages: 0 corresponds to 0%, 0.5 to 50%, and 1 to 100%.

Notation: We write $P(\text{event})$ to mean "the probability of this event happening."

Example: Coin Flip

Question: What is the probability of getting heads when you flip a fair coin?

Answer:

$$P(\text{heads}) = 0.5 = 50\%$$
$$P(\text{tails}) = 0.5 = 50\%$$

Why? Because the coin has 2 equally likely outcomes and heads are 1 of them, so: $\frac{1}{2} = 0.5$.

Example: Rolling a Die

Question: What is the probability of rolling a 6 on a fair die?

Answer:

$$P(\text{rolling a 6}) = \frac{1}{6} \approx 0.167 = 16.7\%$$

Why? Because the die has 6 equally likely outcomes (1, 2, 3, 4, 5, 6), and we want just 1 of them (the 6).

The "Given" Symbol: |

The vertical bar | is one of the most important symbols you'll see in this book!

$P(A|B)$ is pronounced "probability of A *given* B." It means: "What is the probability of A

happening, *if we already know* that B happened?" This is called **conditional probability** because we are computing probability under a certain condition.

Think of the | symbol as a "knowledge divider":

$$P(\text{What we want to know} \mid \text{What we already know})$$

Everything **before** | is what we are trying to predict. Everything **after** | is the information we have.

Example: Weather and Umbrella

Scenario: You see someone carrying an umbrella.

Question 1: What is the probability that it is raining?

$$P(\text{raining}) = 0.3 \text{ (30\% of days are rainy)}$$

Question 2: What is the probability that it is raining, *given* that you see someone with an umbrella?

$$P(\text{raining} \mid \text{umbrella}) = 0.8 \text{ (80\%, much higher!)}$$

Why the difference?

- Without any information: rain is moderately likely (30%)
- *Given* that you see an umbrella: rain is very likely (80%)
- The umbrella gives us additional information that changes our belief!

Example: Language Model Context

For language models, this is EVERYTHING!

$P(\text{next word} \mid \text{all previous words})$

This reads: "Probability of the next word, *given* all the words we have seen so far."

Example sentence: "The cat sat on the _ _ _".

$$P(\text{“mat”} \mid \text{“The cat sat on the”}) = 0.35 \text{ (35\% likely)}$$

$$P(\text{“floor”} \mid \text{“The cat sat on the”}) = 0.25 \text{ (25\% likely)}$$

$$P(\text{“moon”} \mid \text{“The cat sat on the”}) = 0.001 \text{ (very unlikely!)}$$

The model uses the context (everything before |) to predict what comes next!

Practice reading the | symbol:

Notation	Read as…
$P(A\|B)$	“Probability of A given B”
$P(y\|x)$	“Probability of y given x”
$P(w_{\text{next}}\|w_1, w_2, w_3)$	“Probability of next word given words 1, 2, and 3”
$\pi(a\|s)$	“Probability of action a given state s”

The key idea: Everything after the | is “what we know” and helps us predict what’s before the |!

Logarithms: Turning Multiplication into Addition

What Is a Logarithm?

A **logarithm** (written as log) is the inverse of exponential growth. Think of it this way:

- **Exponential:** $2^3 = 8$ means “2 multiplied by itself 3 times equals 8”
- **Logarithm:** $\log_2(8) = 3$ means “to what power do I raise 2 to get 8?”

In this book, log **always means the *natural logarithm*** (base $e \approx 2.718$), also written as ln.

Properties of logarithms (these are super useful!):

1. **Log of 1 is always 0:** $\log(1) = 0$. Why? Because $e^0 = 1$.
2. **Log of a product = sum of logs:**

$$\log(A \times B) = \log(A) + \log(B)$$

   This turns multiplication into addition (easier!).

3. **Log of a quotient = difference of logs:**

$$\log\left(\frac{A}{B}\right) = \log(A) - \log(B)$$

4. **Log of a power:**

$$\log(A^n) = n \cdot \log(A)$$

Key values to remember:

Expression	Value
$\log(1)$	0
$\log(e)$	1 (where $e \approx 2.718$)
$\log(0.5)$	≈ -0.693 (negative because $0.5 < 1$)
$\log(2)$	≈ 0.693
$\log(10)$	≈ 2.303

Why Do We Use Logarithms?

Reason 1: Probabilities multiply, logs add. When we have multiple independent events:

$$P(\text{both}) = P(\text{event 1}) \times P(\text{event 2})$$

But multiplication with tiny numbers (like 0.0001×0.0001) gets messy. Logarithms turn

this into addition:

$$\log P(\text{both}) = \log P(\text{event 1}) + \log P(\text{event 2})$$

Reason 2: Measures "surprise." The logarithm of a probability measures how "surprising" an event is:

- High probability (0.9) → Low surprise → $\log(0.9) = -0.105$ (small negative)
- Medium probability (0.5) → Medium surprise → $\log(0.5) = -0.693$
- Low probability (0.1) → High surprise → $\log(0.1) = -2.303$ (large negative)

We use $-\log$ to flip it so higher surprise = higher value.

Reason 3: Numerical stability. Computers can handle addition better than multiplication of very small numbers. Logs prevent numerical underflow.

Example: Log Probability for a Sentence

Sentence: "The cat sat" (3 words)

Probabilities:

$$P(\text{"The"}) = 0.4$$
$$P(\text{"cat"} \mid \text{"The"}) = 0.3$$
$$P(\text{"sat"} \mid \text{"The cat"}) = 0.5$$

Method 1: Direct multiplication

$$P(\text{sentence}) = 0.4 \times 0.3 \times 0.5 = 0.06$$

Method 2: Log probabilities (better!)

$$\begin{aligned}\log P(\text{sentence}) &= \log(0.4) + \log(0.3) + \log(0.5) \\ &= -0.916 + (-1.204) + (-0.693) = -2.813\end{aligned}$$

Both give the same answer, but Method 2 avoids very small numbers, uses addition (faster for computers), and measures total "surprise" of the sentence.

Expected Value: The Average Outcome

What Is Expected Value?

Expected value (written as $\mathbb{E}[\cdot]$ or sometimes just $E[\cdot]$) is the average value you'd get if you repeated something many times.

Formula:

$$\mathbb{E}[X] = \sum_{\text{all outcomes}} P(\text{outcome}) \times \text{value of outcome}$$

Think of it as: "If I did this 1000 times, what would my average result be?"

Example: Dice Roll Expected Value

Question: What's the expected value when rolling a fair die?

Answer:

$$\begin{aligned}\mathbb{E}[\text{die}] &= P(1) \times 1 + P(2) \times 2 + P(3) \times 3 + P(4) \times 4 + P(5) \times 5 + P(6) \times 6 \\ &= \frac{1}{6} \times 1 + \frac{1}{6} \times 2 + \frac{1}{6} \times 3 + \frac{1}{6} \times 4 + \frac{1}{6} \times 5 + \frac{1}{6} \times 6 \\ &= \frac{1+2+3+4+5+6}{6} = \frac{21}{6} = 3.5\end{aligned}$$

Interpretation: If you roll the die many times, your average result will be 3.5! (You can never roll 3.5 on a single roll, but the average over many rolls is 3.5.)

Example: Reward in Reinforcement Learning

Scenario: An AI generates responses and gets rewards:

Response quality	Probability	Reward
Excellent	20%	+10
Good	50%	+5
Mediocre	25%	0
Bad	5%	−5

Expected reward:

$$\begin{aligned}\mathbb{E}[\text{reward}] &= 0.20 \times 10 + 0.50 \times 5 + 0.25 \times 0 + 0.05 \times (-5) \\ &= 2 + 2.5 + 0 - 0.25 = 4.25\end{aligned}$$

Meaning: On average, this AI gets +4.25 reward per response. The goal of training is to increase this expected value!

Expected Value with Conditions

Conditional Expected Value

Formal Math	What It Really Means
$\mathbb{E}_{X \sim P}[f(X)]$	Expected value of function f when X follows distribution P
$\mathbb{E}[\cdot]$	Expected value (average)
$X \sim P$	Variable X is drawn from distribution P
$f(X)$	We're averaging the values of $f(X)$, not X itself

Example: Language Model Expected Loss

When training a language model:

$$\mathbb{E}_{(x,y)\sim\mathcal{D}}[\text{loss}(x, y)]$$

This reads as: "Expected loss when we sample prompt-response pairs (x, y) from our dataset $\mathcal{D}$."

In plain English: "Average loss across all examples in our training data."

Summation Notation: Adding Things Up

The $\sum$ Symbol

The symbol $\sum$ (capital Greek letter sigma) means "add up all of these things."

Basic form:

$$\sum_{i=1}^{n} x_i = x_1 + x_2 + x_3 + \cdots + x_n$$

Reading: "Sum from i equals 1 to n of x_i." The letter i is the **index** that counts from the starting value (1) to the ending value (n).

Example: Simple Sum

$$\sum_{i=1}^{5} i = 1 + 2 + 3 + 4 + 5 = 15$$

$$\sum_{i=1}^{4} 2i = 2(1) + 2(2) + 2(3) + 2(4) = 2 + 4 + 6 + 8 = 20$$

Example: Sum in Machine Learning

Computing loss for a sequence:

$$\text{Total loss} = \sum_{t=1}^{T} \ell_t$$

This means: "Add up the loss at each time step from $t = 1$ to $t = T$."

If $T = 4$ and losses are: $\ell_1 = 0.5$, $\ell_2 = 0.3$, $\ell_3 = 0.8$, $\ell_4 = 0.4$:

$$\sum_{t=1}^{4} \ell_t = 0.5 + 0.3 + 0.8 + 0.4 = 2.0$$

Infinity in Summation

Sometimes we sum "forever":

$$\sum_{t=0}^{\infty} r_t$$

This means: "Add up all rewards from time 0 to infinity."

Don't panic! In practice, we use a *discount factor* that makes future terms tiny, or we stop after a finite number of steps, or the series converges to a finite value.

Functions and Notation

Functions as Black Boxes

A **function** is like a machine: you put something in (the input), it does some computation, and you get something out (the output).

Notation: $f(x)$ means "function f applied to input x."

Example functions:

- $f(x) = 2x$: doubles the input
- $g(x) = x^2$: squares the input
- $h(x) = \log(x)$: takes the log of the input

The Sigmoid Function

The Sigmoid Function σ

Formal Math	What It Really Means
$\sigma(z) = \frac{1}{1+e^{-z}}$	Squash any number to range [0, 1]
$\sigma(0) = 0.5$	Zero input gives 50% output
$\sigma(+\infty) = 1$	Very large positive input $\to$ 1
$\sigma(-\infty) = 0$	Very large negative input $\to$ 0

Sigmoid in action:

Input z	$\sigma(z)$	**As percentage**
-5	0.007	0.7%
-3	0.047	4.7%
-1	0.269	26.9%
0	0.500	50.0%
$+1$	0.731	73.1%
$+3$	0.953	95.3%
$+5$	0.993	99.3%

Why it is useful: The sigmoid converts any real number to a probability, it is smooth and differentiable (good for training), and it is used everywhere in neural networks and RL!

Loss Functions: Measuring Mistakes

What Is a Loss Function?

A **loss function** (written as $\mathcal{L}$ or L) measures "how wrong" our model is. The key idea: lower loss means better model, and higher loss means worse model. Training a model means finding parameters that *minimize* the loss.

Think of loss like a golf score; your score should be as low as possible, and each mistake increases it.

Common Loss Functions

Negative Log Likelihood Loss

Formal Math	What It Really Means
$\mathcal{L} = -\log P(\text{correct})$	Loss = Negative log probability of correct answer

Why this makes sense:

- If model is confident in correct answer ($P = 0.99$), loss is small ($-\log(0.99) = 0.01$).
- If model is unsure ($P = 0.5$), loss is larger ($-\log(0.5) = 0.69$).
- If model is wrong ($P = 0.01$), loss is huge ($-\log(0.01) = 4.6$).

Example: Loss for Word Prediction

Model predicts next word probabilities:

Word	Probability	Loss if correct
"cat"	0.8	$-\log(0.8) = 0.22$ (good!)
"dog"	0.15	$-\log(0.15) = 1.90$ (okay)
"airplane"	0.01	$-\log(0.01) = 4.61$ (bad!)

If the correct word was "cat": Loss = 0.22. **and if the correct word was "airplane":** Loss = 4.61 (model was very wrong!). Training adjusts the model to reduce these losses.

Gradients and Optimization

What Is a Gradient?

A **gradient** tells you which direction to go in to reduce the loss.

Analogy: Imagine that you are on a foggy mountain trying to get to the lowest point (valley). The gradient tells you which direction is "downhill," you take a small step in that direction, and repeat until you reach the bottom. This process is called **gradient descent**.

In machine learning:

- The "mountain" is the loss function
- The "location" is your model parameters
- The "valley" is the best parameters (minimum loss)

The Training Loop

How gradient descent works:

1. **Start** with random parameters θ
2. **Compute loss** $\mathcal{L}(\theta)$ on training data
3. **Compute gradient** $\nabla_\theta \mathcal{L}$ (which direction reduces loss?)
4. **Update parameters:**

$$\theta_{\text{new}} = \theta_{\text{old}} - \alpha \cdot \nabla_\theta \mathcal{L}$$

   where α is the *learning rate* (step size)
5. **Repeat** steps 2–4 many times
6. **Done** when loss stops decreasing

The gradient ∇_θ just means: "How does loss change when I change each parameter?"

Notation Reference Card

The following table summarizes all the mathematical notation used throughout this book. Recall this table whenever you encounter an unfamiliar symbol.

Symbol	Name	Meaning
$P(A)$	Probability	Chance of event A
$P(A\|B)$	Conditional probability	Probability of A given B
$\|$	"Given"	Separates condition from prediction
$\log(x)$	Logarithm	Inverse of exponential
$\mathbb{E}[X]$	Expected value	Average value of X
$\sum_{i=1}^{n}$	Summation	Add from $i = 1$ to $i = n$
$\mathcal{L}$	Loss function	Measure of model error
θ	Parameters	Model weights (billions of numbers)
π	Policy	Decision-making strategy
$\sigma(z)$	Sigmoid	Squash to [0,1] range
∇	Gradient	Direction of steepest increase
$\sim$	"Drawn from"	Sample from distribution
$\in$	"Element of"	Belongs to a set
$\approx$	Approximately	Roughly equal

The visual cheat sheet below brings all of these ideas together in one place – keep it handy as you work through the rest of the book.

You now have all the mathematical tools you need!

Every equation in this book uses these concepts. When you see complex formulas later, remember: break them down into pieces, identify the symbols from this chapter, read the plain English explanations, and work through the numerical examples.

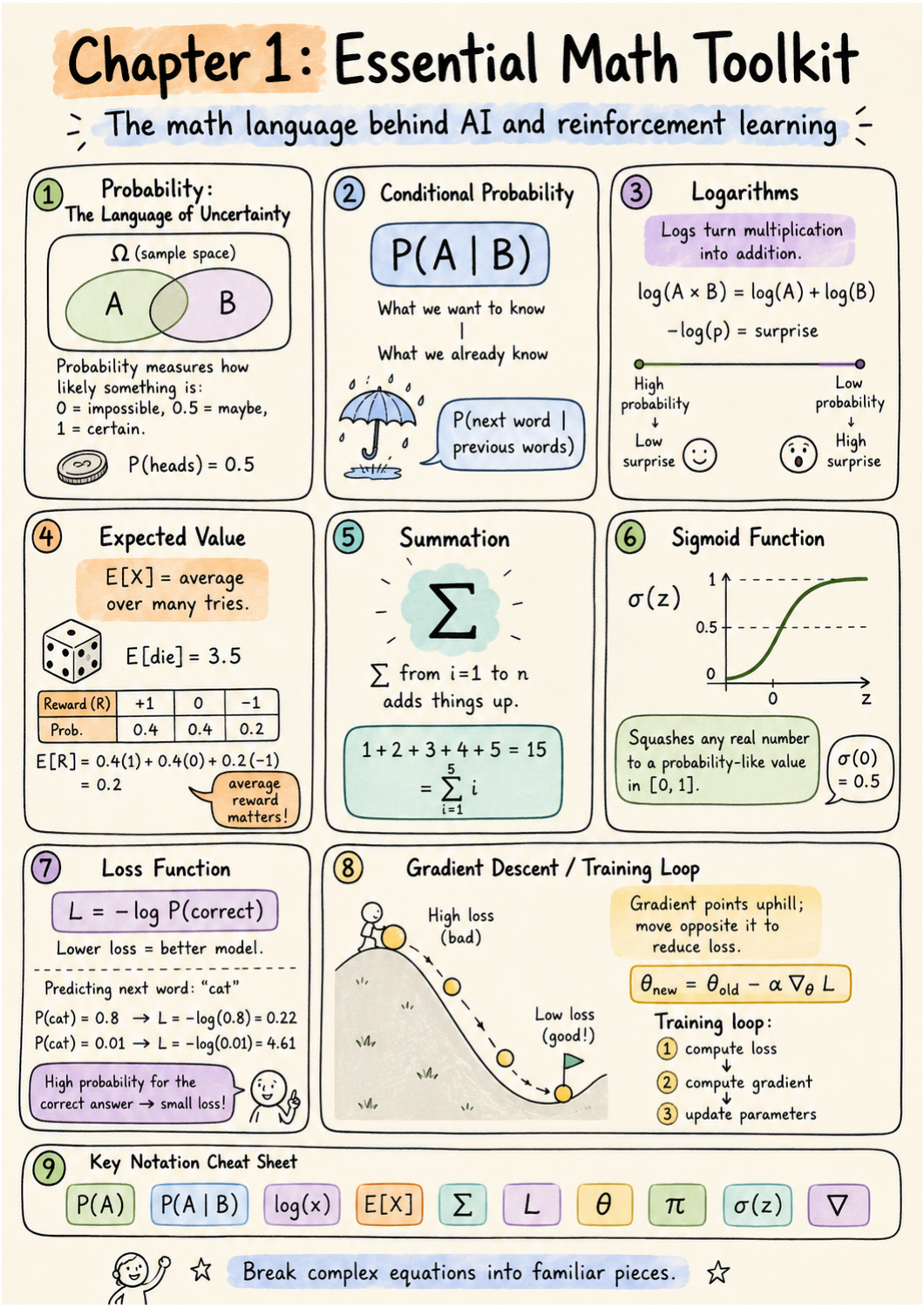

Figure 1.1: Visual cheat sheet for Chapter 1 – probability, conditional probability, logarithms, expected value, summation, sigmoid, loss functions, gradient descent, and key notation at a glance.

You're ready for the main content!

Companion notebook. The code for this chapter is in `code/notebooks/part1_foundations/` in the book repository at `github.com/arunpshankar/packt-final`. On GitHub, navigate to the file and replace `github.com` with `githubtocolab.com` in the URL to open it directly in Colab.